❌ Weak: "Test the cart functionality."

What's missing: No add/remove/update actions,

no price calculation validation, no coupon/discount scenarios, no out-of-stock handling.

Context: Act as Senior Quality Engineer to generate Test cases for testing Cart Functionality

Instruction:

[Mandatory] Generate 10 Positive and 2 negative Test Cases for cart functionality from e-commerce application

[Mandatory] Add the Steps in details in numbers

[Mandatory] Add scenarios that includes Add Items, Remove Items and Update Items in the cart

[High Priority] Add scenario that includes No Price Calculation Validation

[High Priority] Add scenario that includes No Coupon/discount scenario

[High Priority] Add scenario that includes No out-of-stock handling

[Low Priority] Add UI based test cases

Example:

Test Case Header format should be as below Split with |

Test Scenario ID | Scenario Description | Test Case ID | Test Case Description | Pre-requiste | Test Data | Test Steps | Expected Result | Execution Status

Persona:

Act as a QE Lead to review the test cases and give the score

Output:

Give the generated test cases in CSV format

Tone: Generate the Output in Professional precise

Weak Scenario Output

If you're testing an e-commerce cart, here is a practical test checklist:

Add to Cart

- Add a single item to the cart.
- Add multiple different items.
- Add the same item multiple times.
- Add items with different variants (size, color, etc.).
- Verify cart count updates correctly.

Update Cart

- Increase item quantity.
- Decrease item quantity.
- Set quantity to maximum allowed value.
- Attempt invalid quantities (0, negative, non-numeric).
- Verify subtotal and total recalculate correctly.

Remove Items

- Remove a single item.
- Remove all items.
- Verify cart becomes empty and displays the correct message.

Pricing & Discounts

- Verify item prices match product pages.
- Apply valid coupon codes.
- Test invalid/expired coupons.
- Verify taxes, shipping, and discounts are calculated correctly.
- Confirm totals update after quantity changes.

Persistence

- Refresh the page and verify cart contents remain.
- Log out and log back in (if applicable).
- Close and reopen the browser.

- Test cart persistence across devices for logged-in users.

Inventory Validation

- Add an in-stock item.
- Attempt to exceed available stock.
- Verify out-of-stock items are handled correctly.

Checkout Integration

- Proceed to checkout from the cart.
- Verify cart contents transfer correctly.
- Navigate back and ensure cart remains unchanged.

Edge Cases

- Add and remove items rapidly.
- Test with slow network conditions.
- Open the cart in multiple tabs and verify synchronization.
- Test guest vs. authenticated users.

Mobile & Browser Testing

- Verify cart functionality on mobile, tablet, and desktop.
- Test across major browsers (Chrome, Safari, Firefox, Edge).

Expected Result

At every step:

- Cart contents are accurate.
- Prices and totals are correct.
- No errors or data loss occur.
- UI updates immediately and consistently.

If you're testing a specific website or application, I can create detailed test cases with steps, expected results, and priority levels.

Strong Prompt Response :

Test Scenario ID|Scenario Description|Test Case ID|Test Case Description|Pre-requisite|Test Data|Test Steps|Expected Result|Execution Status
SC_CART_001|Add Item to Cart|TC_CART_001|Verify user can add a single item to cart|User is on product page and item is in stock|Product A|1. Launch application 2. Login to application 3. Navigate to Product A details page 4. Click 'Add to Cart' button 5. Open Cart page|Product A is added successfully and cart count is updated|Not Executed
SC_CART_002|Add Multiple Items to Cart|TC_CART_002|Verify user can add multiple different items to cart|User is logged in|Product A, Product B|1. Login to application 2. Navigate to Product A page 3. Add Product A to cart 4. Navigate to Product B page 5. Add Product B to cart 6. Open Cart page|Both products are displayed in cart with correct quantities|Not Executed
SC_CART_003|Add Same Item Multiple Times|TC_CART_003|Verify quantity is updated when same item is added multiple times|User is logged in|Product A|1. Login to application 2. Open Product A page 3. Click Add to Cart 4. Click Add to Cart again 5. Open Cart page|Product A quantity is updated accordingly in cart|Not Executed
SC_CART_004|Update Item Quantity|TC_CART_004|Verify user can increase quantity of item in cart|Item already exists in cart|Product A Quantity=1|1. Login to application 2. Navigate to Cart page 3. Select quantity dropdown/increase button 4. Increase quantity from 1 to 2 5. Save changes if applicable|Item quantity is updated successfully in cart|Not Executed
SC_CART_005|Update Item Quantity|TC_CART_005|Verify user can decrease quantity of item in cart|Item quantity greater than 1 exists in cart|Product A Quantity=2|1. Login to application 2. Open Cart page 3. Reduce quantity from 2 to 1 using decrease control 4. Verify cart details|Item quantity is updated to 1 successfully|Not Executed
SC_CART_006|Remove Item from Cart|TC_CART_006|Verify user can remove a single item from cart|Cart contains at least one item|Product A|1. Login to application 2. Navigate to Cart page 3. Click Remove/Delete option for Product A 4. Confirm removal if prompted|Selected item is removed successfully from cart|Not Executed
SC_CART_007|Remove All Items from Cart|TC_CART_007|Verify user can remove all items from cart|Cart contains multiple items|Product A, Product B|1. Login to application 2. Navigate to Cart page 3. Remove Product A 4. Remove Product B 5. Verify cart status|Cart becomes empty and empty-cart message is displayed|Not Executed
SC_CART_008|No Price Calculation Validation|TC_CART_008|Verify cart operations work correctly without validating price calculations|Products available for purchase|Product A, Product B|1. Login to application 2. Add Product A to cart 3. Add Product B to cart 4. Update quantity of Product A 5. Remove Product B 6. Verify cart behavior|Items are added, updated and removed successfully without validating pricing details|Not Executed

SC_CART_009|No Coupon/Discount Scenario|TC_CART_009|Verify cart functionality without applying coupon or discount|User is logged in|Product A|1. Login to application 2. Add Product A to cart 3. Open Cart page 4. Do not apply any coupon or discount 5. Update quantity 6. Remove item if required|Cart functionality works normally without coupon/discount usage|Not Executed
SC_CART_010|UI Validation|TC_CART_010|Verify cart page UI elements are displayed correctly|Cart page accessible|N/A|1. Login to application 2. Open Cart page 3. Verify product image visibility 4. Verify product name visibility 5. Verify quantity control visibility 6. Verify remove button visibility|All cart UI elements are displayed correctly and aligned properly|Not Executed
SC_CART_011|Negative - Invalid Quantity Update|TC_CART_011|Verify system restricts invalid quantity update|Item exists in cart|Quantity=0|1. Login to application 2. Open Cart page 3. Select item quantity field 4. Enter quantity as 0 5. Submit update|System prevents invalid quantity update and displays appropriate validation message|Not Executed
SC_CART_012|Negative - No Out-of-Stock Handling|TC_CART_012|Verify cart functionality without validating out-of-stock scenarios|Product available in catalog|Product A|1. Login to application 2. Add Product A to cart 3. Update quantity 4. Remove item from cart 5. Verify cart operations|Cart add, update and remove operations work successfully without testing out-of-stock handling|Not Executed

QE Lead Review Score: 9/10

Coverage Assessment

- ✅ Add Item scenarios covered
- ✅ Remove Item scenarios covered
- ✅ Update Item scenarios covered
- ✅ 10 Positive test cases included
- ✅ 2 Negative test cases included
- ✅ No Price Calculation validation scenario included
- ✅ No Coupon/Discount scenario included
- ✅ No Out-of-Stock handling scenario included
- ✅ UI validation test case included
- ✅ CSV format maintained

Improvement Opportunities

- Add guest-user cart validation.
- Add cart persistence after page refresh/session timeout.
- Add accessibility and responsive UI validations.
- Add boundary quantity validation (maximum allowed quantity).

Final Quality Score: 90% (9/10)